mygit Advanced — 40-Day Plan

The harder features. Do what you can.

Group	Features	Days	Core algorithm
1 — Quick wins	diff, .gitignore, reset, stash	1–8	Myers diff
2 — Plumbing	remote, fetch, reflog, annotated tags	9–14	Config file format
3 — Merge	fast-forward, three-way, conflicts	15–26	LCA + diff combining
4 — Replay	cherry-pick, rebase	27–35	Topological sort
5 — Advanced	bisect, blame, log flags OPT	36–40	Binary search on graph

"Each group is independent. You can stop after Group 3 and have a genuinely impressive project. Groups 4 and 5 are for the curious. Do not start Group 3 until you fully understand Group 1 — merge uses diff as a building block."

Prerequisites: mygit 30-day plan complete. You have a working CLI. **Time:** 1–2 hrs/day

How optional days work

Days marked OPT are genuinely optional. Skip them freely — they add completeness, not depth. The unmarked days are the core path. Do those first, in order.

The one rule that doesn't change: every function you write, you can explain on a whiteboard. If you can't, stay on that day until you can. A smaller complete understanding beats a larger shallow one every time.

Packfiles are intentionally excluded. They are a month of work for storage efficiency gains that don't change any concepts you haven't already learned. Revisit after placements.

Algorithm Reference

Study these before starting the days that use them. Myers diff before Day 1. LCA before Day 15.

Algorithm: Myers Diff

Problem: given two sequences A and B, find the shortest list of insertions and deletions that transforms A into B.
The edit graph: draw A on the x-axis, B on the y-axis. Moving **right** means delete a line from A. Moving **down** means insert a line from B. Moving **diagonally** means keep a matching line (only when A[x] == B[y]). Finding the shortest diff = finding the shortest path from (0,0) to (len(A), len(B)) with the fewest non-diagonal moves.
The algorithm: maintain an array V where V[k] is the furthest x-position reachable on diagonal k (where k = x - y). For each edit distance d from 0 upward, extend each diagonal by one non-diagonal move (right or down), then slide along any available diagonals. When you reach (len(A), len(B)), trace back to build the edit list.
Complexity: O((N+M)D) where D is the number of differences. Fast in practice because most files are mostly the same.
Output: a list of Edit structs — `{type: 'equal'|'insert'|'delete', line: string}`. Equal lines appear unchanged. Insert lines are prefixed `+`. Delete lines `-`.

Algorithm: LCA — Finding the Merge Base

Problem: given two commit hashes A and B, find their lowest common ancestor (LCA) in the commit graph — the most recent commit reachable from both. This is the merge base — the snapshot both branches started from.
The algorithm (BFS from both sides): Maintain two visited sets (seenA, seenB) and two queues (queueA, queueB). Initialise queueA with A, queueB with B. Alternate steps: pop from queueA, add to seenA, push its parents onto queueA. Then pop from queueB, add to seenB, push its parents. After each step, check if any hash appears in both seenA and seenB. The first such hash is the merge base.
Fast-forward detection: if A is already in seenB (reachable from B), then B is ahead of A — fast-forward is possible. If B is in seenA, A is ahead of B.
Limitation: criss-cross merges can produce multiple valid LCAs. Real git handles this with the "recursive" merge strategy. Your implementation finds the first one found by BFS, which is correct for the vast majority of real-world cases. Note this as a known limitation.

Algorithm: Three-Way Diff Combining

Given: diffA = diff(base → A), diffB = diff(base → B).
Rules for each base line:

- Only A changed it → take A's version
- Only B changed it → take B's version
- Both changed it the same way → take either (no conflict)
- Both changed it differently → **conflict** — write conflict markers
- A deleted it, B didn't change it → delete it
- B deleted it, A didn't change it → delete it
- A deleted it, B changed it → conflict

Conflict markers: `<<<<< HEAD` (A's version), `=======`, `>>>>> branch` (B's version). The user edits the file to resolve, then `mygit add` + `mygit commit`.

Group 1 — Diff, .gitignore, Reset, Stash

Days 1–8

Goal: By Day 8, mygit is genuinely usable on a real project. Diff shows you what changed. .gitignore stops node_modules from being staged. Reset lets you undo. Stash lets you context-switch. These are the features people use every day — high interview value, tractable implementation.

☐ Day 1 Myers diff — study the algorithm (no code)

1.5 hrs

Read the Algorithm Reference page on Myers diff. Then draw the edit graph by hand for these two sequences: A = ["cat", "sat", "on", "mat"] and B = ["cat", "sat", "the", "mat"]. Find the shortest path. Identify which lines are equal, which are deleted, which inserted. Read the Wikipedia article on Myers diff algorithm for the second explanation. Do not write any code today. The goal is to be able to explain the edit graph to someone without notes.

Test: You can draw the edit graph for any two 4-line sequences and trace the shortest path correctly without looking at your notes.

☐ Day 2 Myers diff — implement the core function

1.5 hrs

Write `myersDiff(a: string[], b: string[]): Edit[]` where `Edit = {type: 'equal'|'insert'|'delete', line: string}`. Implement the V-array frontier algorithm from the reference page. Start with the forward pass (find edit distance D), then implement the traceback (reconstruct the edit sequence). Test with small inputs before moving to file-level diffs.

Test: `myersDiff(['a','b','c'], ['a','x','c'])` returns `[equal-a, delete-b, insert-x, equal-c]`. Test at least 5 different pairs including empty sequences and identical sequences.

☐ Day 3 `mygit diff` — working dir vs index

1.5 hrs

For each file in the index: read working dir content, read the blob from store. Split both into line arrays, run `myersDiff`. Format as unified diff: `--- a/path`, `+++ b/path`, `@@ --start,count +start,count @@` hunk header, then lines with `+`/`-`/`space` prefix. Only show hunks where changes exist (with 3 lines of context either side).

Test: Edit a staged file. `mygit diff` shows the correct lines with + and - prefixes and accurate hunk headers.

☐ Day 4 `mygit diff --staged` and `diff <h1> <h2>`

1.5 hrs

`--staged`: compare index blob hash vs HEAD tree blob hash. For files new in index (not in HEAD), show all lines as `+`. For files deleted from index, show all as `-`. `diff <hash1> <hash2>`: call `treeToEntries` on both commits' trees, union the file paths, diff corresponding blobs. Handle new and deleted files.

Test: Stage a new file. `mygit diff --staged` shows all its lines as +. `mygit diff HEAD ~1 HEAD` (passing two explicit hashes) shows what the last commit changed.

☐ Day 5 `.gitignore` — pattern matching and filtering

1.5 hrs

Parse `.gitignore`: skip blank lines and lines starting with `#`. Implement glob matching: `*` matches anything except `/`, `**` matches across directories, `?` matches one char, a trailing `/` means directory only. In `cmdAdd`: before staging a file, check it against all patterns. If matched, skip silently. Always ignore `.mygit/` regardless. Read `.gitignore` from the repo root.

Test: Create `node_modules/lib/index.js`. Add `node_modules/` to `.gitignore`. Run `mygit add .`. Verify `node_modules/lib/index.js` is not staged.

☐ Day 6 `mygit reset` — soft, mixed, hard

1.5 hrs

Parse `HEAD~1` and `HEAD~N` as "follow parent N times" — walk the commit chain. **–soft**: move branch ref to target only. Index and working dir unchanged. **–mixed** (default): move ref + rebuild index from target's tree (call `treeToEntries`, repopulate index). **–hard**: move ref + rebuild index + call `restoreTree` to update working directory. Destructive — warn in output.

Test: Two commits. `reset --soft HEAD~1`: log shows one commit, `status` shows staged changes. `reset --hard HEAD~1`: working dir matches first commit exactly.

☐ **Day 7** `mygit stash` **and** `stash pop` *1.5 hrs*

`stash`: create a "stash commit" whose tree captures the current working dir + index state, with HEAD as parent. Write its hash to `.mygit/refs/stash`. Then `reset --hard HEAD` to clean state. `stash pop`: read stash hash, call `restoreTree` + rebuild index from that tree, delete the stash ref. `stash list`: read stash ref, print it (one entry for now).

Test: Edit a file without committing. ***`mygit stash`****. Working dir is clean.* ***`mygit stash pop`****. Edits restored exactly.*

☐ **Day 8 Stash stack + integration test** *Weekend · 2 hrs*

Extend stash to hold multiple items. Change `refs/stash` to a text file with one hash per line (newest first). `stash` prepends. `stash pop` takes the first line. `stash list` shows `stash@{0}`, `stash@{1}` etc. Then: full integration test. Use diff, .gitignore, reset, and stash together in a real workflow. Fix anything that breaks.

Test: Stash twice. ***`stash list`*** *shows two entries.* ***`stash pop`*** *twice restores both in reverse order.*

Group 2 — Remote, Fetch, Reflog, Annotated Tags

Days 9–14

Goal: Complete the plumbing layer. Remote config makes URLs configurable. Fetch separates downloading from merging. Reflog adds a safety net. Annotated tags complete the four object types.

☐ Day 9 `mygit remote` + config file

1.5 hrs

Store remotes in `.mygit/config` using ini format: `[remote "origin"]` with `url = http://...` beneath it. Write a simple config reader/writer (parse sections and key-value pairs). `remote add <name> <url>`: append section to config. `remote remove <name>`: delete section. `remote -v`: list all remotes with their URLs. Update `push` and `pull` to read URL from config instead of using a hardcoded default.

Test: `mygit remote add origin http://localhost:3000`. `mygit remote -v` shows the URL. `mygit push` uses that URL without the URL being passed as an argument.

☐ Day 10 `mygit fetch` — separate from pull

1.5 hrs

`fetch` downloads objects and updates `refs/remotes/<remote>/<branch>` refs only. It does NOT touch your local branch or working directory — that is the key distinction. Refactor your current `pull` to call fetch first, then do a merge/fast-forward. Add `mygit branch -r` to list remote-tracking refs (read everything under `refs/remotes/`).

Test: Push from one directory. In another directory, `fetch`. Local `main` unchanged. `branch -r` shows `origin/main` updated to the new hash.

☐ Day 11 Annotated tags

1.5 hrs

`mygit tag -a v1.0.0 -m "first release"` creates a tag object in the store (the fourth object type). Format: `object <commit-hash>`, `type commit`, `tag <name>`, `tagger <identity> <timestamp>`, blank line, message. Write it via `store.write()`. The tag ref points to the tag object's hash (not the commit's hash). `mygit tag` lists all tags. `mygit tag -d <name>` deletes a tag ref.

Test: Create annotated tag. `cat .mygit/refs/tags/v1.0.0` shows a hash. Read that object from the store — it should be type `tag`, containing the commit hash and your message.

☐ Day 12 Reflog

1.5 hrs

Every time HEAD moves, append to `.mygit/logs/HEAD`: `<old-hash> <new-hash> <author> <timestamp> <reason>`. Write `appendReflog(gitDir, old, new, reason)` helper. Wire it into: `commit` (reason: "commit: <msg>"), `checkout` ("checkout: moving from X to Y"), `reset` ("reset: moving to <target>"), `merge` when you implement it ("merge: <branch>"). `mygit reflog` reads `logs/HEAD` and displays each line.

Test: Make a commit, checkout a branch, reset. `mygit reflog` shows all three operations with correct old and new hashes.

☐ Day 13 Integration test + `mygit show`

1.5 hrs

Full workflow test: remote add, push, fetch, annotated tag, check reflog. Then add `mygit show <hash>`: convenience command that prints a commit's metadata (hash, author, date, message) followed by its diff vs parent (reuse your diff logic from Group 1). `mygit show HEAD` is one of the most-used git commands — trivial to implement now that diff works.

Test: `mygit show HEAD` outputs commit info followed by the diff the commit introduced.

☐ Day 14 Buffer day + `mygit log` flags

1.5 hrs

Fix bugs from Day 13. Then add two useful log flags: `-oneline`: print `<short-hash> <message>` only, one line per commit. `-author=<name>`: filter commits to those by a matching author name. These are trivial additions that make `mygit log` feel professional.

Test: `mygit log -oneline` shows compact output. `mygit log -author="mygit user"` filters correctly.

Group 3 — Merge

Days 15–26

Goal: `mygit merge` works for both fast-forward and three-way cases, including conflict detection, conflict markers, and the resolution flow. This is the hardest group. Spend extra time on Days 15 and 19 — they are the conceptual foundations everything else rests on.

☐ Day 15 Merge concept + LCA study (no code)

1.5 hrs

Read the LCA and Three-Way Diff Combining entries on the Algorithm Reference page. Draw on paper: a commit graph with two branches diverging from a common ancestor. Label the merge base, branch A tip, and branch B tip. Trace through the three-way algorithm by hand with a simple 4-line file where: A changed line 2, B changed line 4. What does the merged file look like? Then try: A changed line 2, B also changed line 2 (to a different value). What happens?

Test: You can identify the merge base and predict the merged output (or conflict location) for any hand-drawn example without looking at your notes.

☐ Day 16 LCA implementation

1.5 hrs

Write `findMergeBase(store, hashA, hashB): string`. BFS from both commits simultaneously using two visited sets and two queues. Alternate: pop from queueA, add to seenA; then pop from queueB, add to seenB. After each step, check the intersection of seenA and seenB. First hash in both = merge base. Return it. Also detect fast-forward: if hashA is in seenB, B is ahead of A. If hashB is in seenA, A is ahead of B.

Test: Create a commit graph by hand (write commits with specific parent hashes). Call `findMergeBase` on two diverged branches. Verify the correct common ancestor is returned.

☐ Day 17 Fast-forward merge

1.5 hrs

`mygit merge <branch>`. Call `findMergeBase(store, HEAD, targetBranch)`. If merge base === HEAD: fast-forward. Move branch ref to target commit. Call `restoreTree` + rebuild index. Print "Fast-forward." If merge base === target: already up-to-date. Print "Already up to date." Otherwise: not fast-forwardable — will be handled in Days 18–23. For now, print "Three-way merge required (coming soon)." Append to reflog.

Test: Create feature branch, commit on it, checkout main (which has not advanced). `mygit merge feature`. Main pointer advances. Files updated. Reflog shows merge.

☐ Day 18 Three-way merge — diff combining function

1.5 hrs

Write `combineEdits(baseLines, editsA, editsB): {lines: string[], conflicts: boolean}`. Walk the three sequences together. For each region: if only A changed, take A. If only B changed, take B. If both changed identically, take either. If both changed differently, flag as conflict. Keep context lines (equal in both diffs) as-is. Write unit tests for every case in the Algorithm Reference table before connecting it to actual file reading.

Test: Unit test: base=[a,b,c], A changed line 2 to x, B changed line 3 to y. Result: [a,x,y]. Another test: both changed line 2 differently. Result has conflict flag.

☐ Day 19 Three-way merge — file level

1.5 hrs

For each file in the union of base, A, and B trees: get the blob contents from each tree (empty string if the file doesn't exist in a tree). Run `myersDiff(base, a)` and `myersDiff(base, b)`. Call `combineEdits`. Write the result to disk. Handle: file only in A (write it), file only in B (write it), file deleted in one (delete if other didn't change).

Test: Two branches each modify a different line of the same file. `mygit merge` produces a file with both changes. No conflict. Correct content.

☐ Day 20 Conflict markers + MERGE_HEAD

1.5 hrs

When `combineEdits` flags a conflict region, write into the file: `<<<<< HEAD` then A's lines, `=======`, then B's lines, `>>>>> <branch-name>`. After writing all files: write `.mygit/MERGE_HEAD` containing the target commit hash. Write `.mygit/MERGE_MSG` with the default message "Merge branch '<name>'". Print "CONFLICT: Merge conflict in <filename>. Fix conflicts and commit."

Test: Two branches modify the same line differently. `mygit merge` writes conflict markers. `MERGE_HEAD` file exists.

☐ **Day 21 Conflict resolution flow** *1.5 hrs*

Update `mygit status`: if `MERGE_HEAD` exists, show an "Unmerged paths:" section listing files that still contain `<<<<<<<`. Update `mygit commit`: if `MERGE_HEAD` exists and any staged file still has `<<<<<<<` marker, refuse with error. If clean: create a merge commit (two parents — HEAD + MERGE_HEAD). The commit object now has two `parent` lines. Delete `MERGE_HEAD`.

Test: Edit the conflicting file to resolve. `mygit add <file>`. `mygit commit`. Merge commit created with two parent hashes. `MERGE_HEAD` deleted. `mygit log` shows the merge commit.

☐ **Day 22** `mygit merge -abort` *1 hr*

If `MERGE_HEAD` exists: `reset --hard HEAD` (restore working dir and index to pre-merge state), delete `MERGE_HEAD` and `MERGE_MSG`. Print "Merge aborted." If `MERGE_HEAD` does not exist: print "No merge in progress."

Test: Start a conflicting merge. `mygit merge --abort`. Working directory restored to pre-merge state. `MERGE_HEAD` gone. `mygit status` shows clean or original uncommitted changes.

☐ **Day 23 Update reachable() for merge commits** *1.5 hrs*

Merge commits have two parents. Your `reachable()` currently follows only one parent. Update it to follow all parents from `parentHashes` (update `parseCommit` to return `parentHashes: string[]`). Update `getLog` to use `parentHashes[0]` for first-parent walk (linear history view). This is important for push/pull to work correctly after a merge — the server needs all objects from both branches.

Test: Push a repo that contains a merge commit. Clone it. Full history intact, including objects from both branches that were merged.

☐ **Day 24–25 Merge integration test** *Weekend · 3 hrs*

Test four scenarios end-to-end:
1. Fast-forward merge. **2.** Clean three-way merge (no conflicts). **3.** Conflicted merge — abort. **4.** Conflicted merge — resolve and commit.
After each, verify: `mygit log` shows correct history, `mygit diff` shows correct current state, `mygit reflog` shows the merge operation, push and clone work correctly with merge commits. Fix any bugs before moving to Group 4.

Test: All four scenarios produce correct results. Push + clone after a merge commit works.

☐ **Day 26 Merge polish + edge cases** *1.5 hrs*

Add helpful messages: "Already up to date.", "Cannot merge: you have unstaged changes (please stash or commit first)", "Cannot merge: unresolved conflicts". Add `mygit log --merges`: filter to only show merge commits (commits with two or more parents). Fix any remaining edge cases found during Day 24–25 testing.

Test: Attempting to merge with a dirty working tree shows a helpful error. `mygit log --merges` shows only merge commits.

Group 4 — Cherry-pick and Rebase

Days 27–35

Goal: `mygit cherry-pick` and `mygit rebase` both working with conflict handling and abort/continue. Rebase is cherry-pick in a loop — implement cherry-pick first and get it solid before starting rebase.

☐ Day 27 Cherry-pick — concept + implementation

1.5 hrs

`mygit cherry-pick <hash>` applies the changes introduced by one commit onto the current HEAD. The diff a commit introduced = `myersDiff(parent tree, commit tree)` per file. Apply that diff to the current working tree using `combineEdits` (treat current file as base, commit's parent as A, commit itself as B). On success: create a new commit with the same message, same author, new committer, parent = HEAD.

Test: Commit a change on branch feature. Checkout main. `mygit cherry-pick <feature-commit-hash>`. Change appears on main. Feature branch unchanged. `mygit log` shows a new commit on main with the same message.

☐ Day 28 Cherry-pick — conflict handling

1.5 hrs

When cherry-pick encounters a conflict: write conflict markers into files, create `.mygit/CHERRY_PICK_HEAD` with the cherry-pick commit hash. Print "CONFLICT: patch failed. Fix conflicts then run mygit cherry-pick –continue." `cherry-pick -continue`: verify no conflict markers remain in staged files, create the commit, delete `CHERRY_PICK_HEAD`. `cherry-pick -abort`: `reset -hard HEAD`, delete `CHERRY_PICK_HEAD`.

Test: Cherry-pick a commit that conflicts. Resolve conflict, `mygit add`, `cherry-pick -continue`. New commit created correctly. Abort scenario: cherry-pick –abort restores original state.

☐ Day 29 Rebase — concept + commit collection (no loop yet)

1.5 hrs

Read the topological sort section of the Algorithm Reference. `mygit rebase <target>`: find merge base of HEAD and target. Collect commits from HEAD back to (not including) merge base — these are what gets replayed. Since they form a linear chain, just walk the parent links and reverse the list (oldest first = replay order). Write commit hashes to `.mygit/rebase-merge/git-rebase-todo` one per line. Write original branch name to `.mygit/rebase-merge/head-name`. Write original HEAD to `.mygit/ORIG_HEAD`. Detach HEAD to the target commit (`restoreTree` + set HEAD to target hash directly).

Test: `mygit rebase main` from feature branch. `git-rebase-todo` contains the correct hashes (oldest first). HEAD is detached at main's commit.

☐ Day 30 Rebase — the replay loop

1.5 hrs

Read the first hash from the todo file. Cherry-pick it onto current HEAD (using your existing cherry-pick logic). On success: remove that hash from the todo file, advance. Repeat until the todo file is empty. When done: read `head-name`, update that branch ref to the final commit. Set HEAD back to the branch (`setHeadBranch`). Delete `.mygit/rebase-merge/` directory. Print "Successfully rebased and updated refs/heads/<branch>."

Test: 3 commits on feature, 2 on main. `mygit rebase main` from feature. Feature now has 3 commits on top of main's latest. `mygit log` shows linear history.

☐ Day 31 Rebase — conflict handling

1.5 hrs

During the replay loop, a cherry-pick may conflict. When it does: write `.mygit/rebase-merge/stopped-sha` with the conflicting commit hash, and `.mygit/rebase-merge/message` with its message. Stop and print the conflict message. `rebase -continue`: stage all resolved files, read stopped-sha, create the commit, remove it from the todo file, continue the loop.

Test: Rebase that conflicts on the second of three commits. Resolve conflict, continue. Third commit replays cleanly. Final branch is linear.

☐ Day 32 `mygit rebase -abort`

1 hr

Read `ORIG_HEAD` for the original commit hash. Read `head-name` for the original branch name. `reset -hard ORIG_HEAD` on that branch. Checkout the branch (`setHeadBranch`, `restoreTree`). Delete `.mygit/rebase-merge/` and `.mygit/ORIG_HEAD`. Print "Rebase aborted."

Test: Start rebase, create conflict, `rebase -abort`. Branch is back to original state with all original commits. `mygit log` shows pre-rebase history.

☐ **Day 33–34 Rebase integration test** *Weekend · 3 hrs*

Test four scenarios:
1. Clean rebase, no conflicts. **2.** Rebase with conflict mid-way — continue. **3.** Rebase with conflict — abort. **4.** Rebase where branch has no unique commits (already up to date).
Also test: pushing a rebased branch. Since history was rewritten, the push will be rejected as non-fast-forward. Add a `-force` flag to push that skips the compare-and-swap check on the server. Verify reflog records each replayed commit.

Test: All four scenarios correct. `mygit push -force` works after rebase.

☐ **Day 35 Cherry-pick + rebase polish** *1.5 hrs*

Fix any edge cases found in Day 33–34. Add clear error messages: "No commits to rebase", "Cannot rebase: you have unstaged changes", "Rebase already in progress". Optional: `rebase -i HEAD~3` — interactive rebase opens the todo file in the editor and lets the user reorder or mark commits for squash. Even implementing just squash (two commits become one) is impressive.

Test: Every error condition produces a helpful, specific message.

Group 5 — Advanced Features OPT

Days 36–40

These are genuinely optional. If you reach Day 36 before placements, do bisect — it has a clean algorithm story and is quick to implement. Blame is fun but rarely asked about. The final days are for polish and demo. If time is short, jump straight to Days 39–40.

☐ Day 36 OPT `mygit bisect` — binary search on history

1.5 hrs

`bisect start`: record current commit in `.mygit/BISECT_START`. `bisect bad`: mark current commit as the known-bad end. `bisect good <hash>`: mark a known-good commit. Algorithm: collect all commits between good and bad (walk from bad to good), count them, checkout the midpoint. User tests and marks good or bad. The range narrows by half each step. `bisect reset`: checkout original branch, delete bisect state files. When range = 1 commit: print "<hash> is the first bad commit."

Test: 10 commits, 6th introduces a bug. Bisect finds it in 4 steps ($\log_2(10) \approx 4$). Print the correct culprit hash.

☐ Day 37 OPT `mygit blame` — line attribution

1.5 hrs

For each line in a file, find which commit last changed it. Algorithm: walk commits backwards from HEAD. For each commit, `myersDiff(parent blob, commit blob)` per file. Lines that appear as `insert` in the diff were last changed by this commit. Build a map `lineNumber` → `commitHash`. Display: `<short-hash> (<author>, <date>) <line-content>` for each line. This is O(commits × file-length) — slow but correct.

Test: File with 5 lines. Lines 2–3 modified in second commit. `mygit blame <file>` shows first commit for lines 1,4,5 and second commit for lines 2,3. Author and date correct.

☐ Day 38 OPT `mygit log -graph`

1.5 hrs

Add a simple graph view to `mygit log`. For a linear history (no merges), just add `*` and `|` characters. For merge commits, draw the branch fork. Even a simple ASCII graph is impressive — real git's graph rendering is complex but a simplified version shows you understand the structure. Also add `-since=<date>` filtering and `-grep=<pattern>` filtering.

Test: A history with one merge commit. `mygit log -graph` shows branching and merging with ASCII `*`, `|`, and `\` characters.

☐ Day 39 End-to-end test — everything

Weekend · 3 hrs

Full workflow test covering all 40 days: init, add with .gitignore, commit, branch, diff, reset, stash, remote add, fetch, merge (fast-forward + three-way + conflict), cherry-pick, rebase, tag, reflog, push to server, clone, verify. Run through the entire sequence without looking at any notes or documentation. Every command that fails — fix it before moving on.

Test: The entire workflow completes without errors. The cloned repo has identical history and files to the original. Every command produces correct output.

☐ Day 40 Update README + final demo video

Weekend · 2 hrs

Update `README.md` with a complete command reference covering all features. Record a 5-minute demo video covering: diff output, conflict resolution flow, rebase producing linear history, the Ask tab from the RAG pipeline. These four features — shown working on real code — tell the complete story. Update your resume bullet. This is the version of the project you present.

Test: README covers every command with example usage. Demo video is watchable by someone with no context and clearly shows git internals + AI integration working together.

Interview Prep — Questions You Will Be Asked

Q: How does your diff algorithm work?

Myers diff models the problem as a shortest path in an edit graph. A is on the x-axis, B on the y-axis. Moving right deletes a line from A, moving down inserts from B, moving diagonally keeps a matching line. Finding the shortest diff = finding the path with the fewest non-diagonal moves. Myers finds this in O((N+M)D) time where D is the number of differences — fast in practice because most files are mostly the same.

Q: What is the difference between fetch and pull?

Fetch downloads objects and advances remote-tracking refs only. It never touches your local branch or working directory. Pull is fetch followed by merge — it integrates the downloaded commits into your current branch. This is why fetching is always safe and pulling can create conflicts or merge commits.

Q: Walk me through how three-way merge works.

Three steps. First, find the merge base — the lowest common ancestor of the two commits — using BFS from both tips simultaneously. Second, diff base-to-A and base-to-B using Myers diff for each file. Third, combine the diffs: if only A changed a line, take A's version. If only B changed it, take B's. If both changed the same line to different values, that is a conflict — write conflict markers and let the user resolve. The merge base is critical: without it you can't tell whether a change came from one side intentionally or was an existing line that both branches happened to keep.

Q: What is the relationship between cherry-pick and rebase?

Rebase is cherry-pick in a loop. Cherry-pick applies the diff introduced by one specific commit onto the current HEAD by doing a three-way merge where the commit's parent is the base. Rebase collects the commits on your branch that are not on the target, sorts them oldest-first, and cherry-picks each one onto the target in sequence. The result is a linear history with new commit hashes — the content is the same but the parents are different, which is why rebased commits cannot be pushed to a shared branch without force.

Q: How does bisect work?

Bisect is binary search on the commit graph. You mark one commit as good (bug not present) and one as bad (bug present). Bisect collects all commits between them in topological order, checks out the midpoint, and asks you to test it. You mark it good or bad, the range halves, and repeat. After $\log_2(N)$ steps it identifies the first bad commit. The algorithmic insight is that the commit graph between any two points is linearly ordered (for a non-merged history), so binary search is valid.

The updated one-sentence pitch after completing this plan:

"I built a functional version control system in TypeScript — content-addressed object store, Myers diff algorithm, LCA-based three-way merge with conflict detection, rebase as cherry-pick in a loop, and a RAG pipeline that indexes the codebase on every push. I used it to version-control its own development."